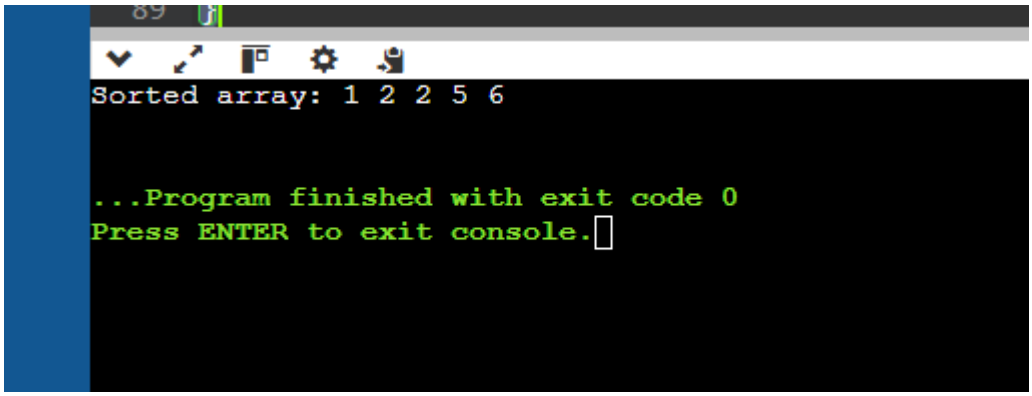
Lab3-**3. Merge sort: Sort an array**

```
#include <stdio.h>
#include <stdlib.h>

void merge(int* nums, int* temp, int left, int mid, int right) {

  int i = left;
  int j = mid + 1;
  int k = left;

  while (i <= mid && j <= right) {

    if (nums[i] <= nums[j]) {
      temp[k] = nums[i];
      i++;
    }
    else {
      temp[k] = nums[j];
      j++;
    }

    k++;
  }

  while (i <= mid) {
    temp[k] = nums[i];
    i++;
```

```
        k++;
    }

    while (j <= right) {
        temp[k] = nums[j];
        j++;
        k++;
    }

    for (i = left; i <= right; i++) {
        nums[i] = temp[i];
    }
}

void mergeSort(int* nums, int* temp, int left, int right) {

    if (left >= right) {
        return;
    }

    int mid = left + (right - left) / 2;

    mergeSort(nums, temp, left, mid);
    mergeSort(nums, temp, mid + 1, right);

    merge(nums, temp, left, mid, right);
}
```

```
int* sortArray(int* nums, int numsSize, int* returnSize) {

    *returnSize = numsSize;

    int* temp = malloc(numsSize * sizeof(int));

    if (temp == NULL) {
        return NULL;
    }

    mergeSort(nums, temp, 0, numsSize - 1);

    free(temp);

    return nums;
}

int main() {

    int nums[] = {6, 2, 5, 2, 1};
    int numsSize = 5;
    int returnSize;

    sortArray(nums, numsSize, &returnSize);

    printf("Sorted array: ");

    for (int i = 0; i < returnSize; i++) {
```

```
        printf("%d ", nums[i]);
    }

    printf("\n");

    return 0;
}
```